Poimano — Documentación Técnica

> **SaaS Multi-Tenant para Gestión de Iglesias** Versión: 1.0.0-alpha · Última actualización: 14 de marzo de 2026

Índice

1. Visión General
2. Stack Tecnológico
3. Arquitectura del Sistema
4. Infraestructura y Despliegue
5. Multi-Tenancy
6. Base de Datos Central
7. Base de Datos Tenant
8. Core SaaS (Plataforma)
9. Sistema Modular (Negocio)
10. Panel Super Admin (Filament)
11. Frontend Tenant (React + Inertia)
12. Autenticación y Autorización
13. Rutas del Sistema
14. Seeders y Datos Iniciales
15. Estado Actual por Módulo
16. Estructura de Archivos
17. Flujos Críticos
18. Roadmap Pendiente

---

1. Visión General

Poimano es una plataforma SaaS diseñada para la gestión integral de iglesias. Permite a cada iglesia (tenant) administrar miembros, familias, grupos, asistencia, finanzas, actividades, comunicaciones, proyectos y reportes desde su propio subdominio.

Propuesta de valor:

- Una iglesia se registra → obtiene su subdominio (`miIglesia.poimano.com`)
- Se le asigna un plan de suscripción con módulos específicos habilitados
- Cada iglesia tiene su base de datos independiente (aislamiento total)
- Un super administrador gestiona todas las iglesias desde un panel centralizado

Modelo de negocio: 4 planes (Gratuito → Enterprise) con módulos desbloqueados progresivamente.

---

2. Stack Tecnológico

Backend

Tecnología	Versión	Función
PHP	8.3	Lenguaje backend
Laravel	11.48	Framework MVC principal
PostgreSQL	16	Motor de base de datos (central + tenants)
stancl/tenancy	3.9	Multi-tenancy database-per-tenant
Filament	3.3	Panel administrativo del super admin
Inertia.js	2.0	Bridge backend-frontend (server-side routing con SPA UX)
spatie/laravel-activitylog	4.12	Auditoría y registro de cambios
predis/predis	3.4	Cliente Redis para cache/session/queue

Frontend

Tecnología	Versión	Función
React	19.2	UI library (tenant frontend)
TypeScript	5.9	Tipado estático para JavaScript
@inertiajs/react	2.3	Adaptador React para Inertia.js
shadcn/ui	—	Componentes UI basados en Radix UI
@tanstack/react-table	8.21	Tablas con sorting, filtering, paginación
Recharts	3.8	Gráficos y visualización de datos
Lucide React	0.577	Sistema de iconos
Tailwind CSS	3.4	Framework CSS utility-first
Vite	6.0	Build tool y dev server con HMR

DevOps / Infraestructura

Tecnología	Función
Ubuntu 24.04 (VPS Hostinger)	Servidor de producción
Nginx	Servidor web / reverse proxy
Cloudflare	DNS + SSL wildcard (`*.poimano.com`)
GitHub	Repositorio de código
Git	Control de versiones y deploy manual

---

3. Arquitectura del Sistema

Patrón Arquitectónico: DDD Lite + MVC + Modular Monolith

```
┌──────────────────────────────────────────────────────────────────────┐
│                          POIMANO PLATFORM                            │
│                                                                      │
│   ┌──────────────────┐          ┌──────────────────────┐            │
│   │  PANEL CENTRAL   │          │   FRONTEND TENANT    │            │
│   │  (Filament PHP)  │          │   (React + Inertia)  │            │
│   │                  │          │                      │            │
│   │  poimano.com     │          │  {slug}.poimano.com  │            │
│   │  /admin          │          │  /login /dashboard   │            │
│   │                  │          │  /members /groups ...  │            │
│   └──────────────────┘          └──────────────────────┘            │
│            │                                  │                      │
│   ┌────────▼──────────────────────────────────▼──────────────┐      │
│   │                  LARAVEL 11 APPLICATION                   │      │
│   │                                                           │      │
│   │   ┌──────────┐   ┌──────────────────────────────┐        │      │
│   │   │  CORE    │   │          MODULES             │        │      │
│   │   │  SaaS    │   │   (DDD Lite por módulo)      │        │      │
│   │   │          │   │                              │        │      │
│   │   │ Tenants  │   │  Church     │ Members ✅     │        │      │
│   │   │ Plans    │   │  Groups     │ Attendance     │        │      │
│   │   │ Billing  │   │  Finance    │ Activities     │        │      │
│   │   │ Settings │   │  Comms      │ Projects       │        │      │
│   │   │          │   │  Reports    │                │        │      │
│   │   └──────────┘   └──────────────────────────────┘        │      │
│   │                                                           │      │
│   │   ┌──────────────────────┐   ┌──────────────────┐        │      │
│   │   │  stancl/tenancy 3.9  │   │  Middleware Stack │        │      │
│   │   │  DB-per-tenant       │   │  SubdomainID      │        │      │
│   │   │  Auto-migrate/seed   │   │  ModuleAccess     │        │      │
│   │   └──────────────────────┘   └──────────────────┘        │      │
│   └──────────────────────────────────────────────────────────┘      │
│                                                                      │
│   ┌──────────────────┐      ┌──────────────────────────┐            │
│   │  PostgreSQL 16   │      │    Redis (Cache/Session) │            │
│   │                  │      │                          │            │
│   │  poimano_central │      │    Prefixed per tenant   │            │
│   │  tenant_iglesia1 │      └──────────────────────────┘            │
│   │  tenant_iglesia2 │                                              │
│   │  tenant_...      │                                              │
│   └──────────────────┘                                              │
└──────────────────────────────────────────────────────────────────────┘
```

Capas de la Arquitectura

Capa	Descripción	Ejemplo
Presentation	Interfaz de usuario (Filament o React/Inertia)	`Members/Index.tsx`, `TenantResource.php`

Capa	Descripción	Ejemplo
Application	Casos de uso, validación, DTOs	`MemberController.php`, `AuthController.php`
Domain	Modelos, reglas de negocio, eventos	`Member.php`, `Plan.php`, `Tenant.php`
Infrastructure	Repositorios, providers, persistencia	`TenancyServiceProvider.php`, migrations

Tres Capas de Datos

1. **Central (compartida):** Datos de la plataforma — tenants, planes, módulos, roles, permisos, países, templates
2. **Shared (tablas centrales accedidas por tenants):** Roles y permisos (definidos centralmente, relacionados desde tenant)
3. **Tenant (aislada por iglesia):** Usuarios locales, miembros, familias, configuración de iglesia, logs de auditoría

---

4. Infraestructura y Despliegue

Servidor de Producción

Componente	Detalle
Proveedor	Hostinger VPS
IP	187.124.157.50
OS	Ubuntu 24.04 LTS
Web Server	Nginx
PHP	8.3 (PHP-FPM)
PostgreSQL	16
Ruta proyecto	`/var/www/poimano`

Dominio y DNS

Componente	Detalle
Dominio principal	poimano.com
DNS	Cloudflare
SSL	Wildcard `*.poimano.com` (Cloudflare)
Panel admin	https://poimano.com/admin
Subdominios tenant	https://{slug}.poimano.com

Flujo de Deploy

```
Local: git push origin main
  ↓
VPS (manual por SSH):
  cd /var/www/poimano
  git pull origin main
  composer install --no-dev
  npm install && npm run build
  php artisan optimize:clear
  php artisan optimize
```

---

5. Multi-Tenancy

Estrategia: Database-per-Tenant (stancl/tenancy 3.9)

Cada iglesia tiene su propia base de datos PostgreSQL. Esto garantiza:

- **Aislamiento total** de datos entre iglesias
- **Escalabilidad** independiente por tenant
- **Seguridad** — imposible acceder datos de otra iglesia
- **Backup/restore** independiente por iglesia

Identificación del Tenant

Mecanismo	Detalle
Tipo	Subdomain-based
Middleware	`InitializeTenancyBySubdomain`
Flujo	`iglesia1.poimano.com` → extrae `iglesia1` → busca en tabla `domains` → inicializa tenant context

Bootstrappers Activos

Bootstrapper	Función
`DatabaseTenancyBootstrapper`	Cambia la conexión de BD activa al tenant
`CacheTenancyBootstrapper`	Prefija las claves de cache con el tenant ID
`FilesystemTenancyBootstrapper`	Aísla `storage/app/` por tenant
`QueueTenancyBootstrapper`	Adjunta tenant context a jobs en cola

Dominios Centrales (no-tenant)

```
'central_domains' => ['127.0.0.1', 'localhost', 'poimano.localhost',
'poimano.com']
```

Estos dominios NO inicializan tenancy y sirven el panel central (Filament) y la landing page.

Ciclo de Vida del Tenant

```
Crear Iglesia (Filament) → Evento TenancyCreated
  ├─ Job: CreateDatabase      → CREATE DATABASE tenant_{slug}
  ├─ Job: MigrateDatabase     → php artisan tenants:migrate (10 rutas de
migración)
  ├─ Job: SeedDatabase        → TenantDatabaseSeeder (roles, settings, admin user)
  └─ afterCreate()            → Crear dominio + usuario admin con password del
formulario
```

---

6. Base de Datos Central

Esquema: `poimano_central`

Tablas de Plataforma (stancl/tenancy)

Tabla	Campos clave	Función
`tenants`	id, church_name, slug, plan_id, status, pastor_name, email, phone, address, data (JSON)	Registro de cada iglesia
`domains`	id, domain, tenant_id	Subdominios asociados a tenants

Tablas de Usuarios y Auth

Tabla	Campos clave	Función
`users`	id, name, email, password, is_admin, role	Super admins de la plataforma
`password_reset_tokens`	email, token	Reset de contraseñas
`sessions`	id, user_id, ip_address, payload	Sesiones activas

Tablas de Planes y Módulos

Tabla	Campos clave	Función
`plans`	id, name, slug, description, price, billing_cycle, max_members, features (JSON), is_active	Planes de suscripción
`modules`	id, name, slug, description, icon, is_active, sort_order	Módulos disponibles del sistema
`plan_modules`	plan_id, module_id	Relación N:M que define qué módulos incluye cada plan

Tablas Compartidas (Shared)

Tabla	Campos clave	Función
`roles`	id, name, slug, scope, description, is_default	Roles RBAC (central y tenant)
`permissions`	id, name, slug, module, description	Permisos granulares por módulo
`role_permission`	role_id, permission_id	Relación N:M roles-permisos
`countries`	id, name, code, phone_code, currency, currency_symbol	Catálogo de países
`templates`	id, name, slug, type, module, subject, body, variables (JSON)	Templates de email/SMS/notificaciones

Tablas de Configuración y Soporte

Tabla	Campos clave	Función
`settings`	key (PK), value	Configuración clave-valor de plataforma (con cache Redis 1h)
`notifications`	id, type, notifiable_id, data (JSONB)	Notificaciones de Laravel
`cache`	key, value, expiration	Cache store (fallback si Redis no disponible)
`jobs` / `job_batches` / `failed_jobs`	—	Cola de trabajos (queue)
`activity_log`	id, log_name, description, subject, causer, properties	Logs de auditoría (Spatie)

7. Base de Datos Tenant

Esquema: `tenant_{slug}` (por cada iglesia)

Tabla: `users` (Usuarios locales de la iglesia)

Campo	Tipo	Descripción
id	bigint PK	Identificador
name	string	Nombre completo
email	string (unique)	Correo electrónico
password	string (hashed)	Contraseña (bcrypt)
role	string (default: 'member')	Rol local: admin, pastor, leader, member

Campo	Tipo	Descripción
phone	string nullable	Teléfono
address	text nullable	Dirección
birth_date	date nullable	Fecha de nacimiento
avatar	string nullable	Ruta de foto de perfil
is_active	boolean (default: true)	Estado activo/inactivo
email_verified_at	timestamp nullable	Verificación de email
remember_token	string	Token de sesión persistente
timestamps	—	created_at, updated_at

Tablas asociadas: `password_reset_tokens`, `sessions`

Tabla: `church_settings` (Configuración de la iglesia)

Campo	Tipo	Descripción
id	bigint PK	Identificador
church_name	string	Nombre de la iglesia
logo	string nullable	Logo de la iglesia
slogan	string nullable	Lema
primary_color	string (#00105E)	Color primario
secondary_color	string (#00E1FF)	Color secundario
language	string (es)	Idioma
currency	string (USD)	Moneda
timezone	string (America/Bogota)	Zona horaria
phone, email, address, city	—	Datos de contacto
country_id	FK nullable	País (referencia central)
website	string nullable	Sitio web
founded_at	date nullable	Fecha de fundación
denomination	string nullable	Denominación

Tabla: `families` (Familias/hogares)

Campo	Tipo	Descripción
id	bigint PK	Identificador

Campo	Tipo	Descripción
name	string	Apellido o nombre de familia
address	text nullable	Dirección del hogar
phone	string nullable	Teléfono familiar
wedding_date	date nullable	Fecha de matrimonio
notes	text nullable	Notas
timestamps + soft_deletes	—	—

Tabla: `members` (Miembros — ~30 columnas)

Sección	Campos	Descripción
Personal	first_name, middle_name, last_name, second_last_name, gender (enum M/F), birth_date, photo, blood_type, marital_status (enum 5 opciones), nationality, document_type, document_number	Datos personales completos
Contacto	email, phone, mobile, address, city, state, zip_code, country_id	Información de contacto
Espiritual	member_status (5 estados), category (5 categorías), is_baptized, baptism_date, baptism_church, conversion_date, membership_date, how_arrived, referred_by_id	Vida espiritual y eclesiástica
Familia	family_id (FK), family_role (cabeza/esposa/hijo/otro)	Vínculo familiar
Sistema	user_id (FK nullable), notes, is_active, soft_deletes, timestamps	Administración

Índices: email, phone, member_status, category, family_id, is_active

Traits: `SoftDeletes`, `LogsActivity` (Spatie — auditoría automática)

Scopes: `active()`, `byCategory()`, `baptized()`

Accessors: `full_name`, `complete_name`

Tabla: `member_skills` (Habilidades/dones)

Campo	Tipo	Descripción
id	bigint PK	Identificador
member_id	FK (cascade delete)	Miembro asociado
skill	string	Habilidad o don (unique por miembro)

Tabla: `member_history` (Historial de cambios de estado)

Campo	Tipo	Descripción
id	bigint PK	Identificador
member_id	FK (cascade delete)	Miembro asociado
from_status	string nullable	Estado anterior
to_status	string	Estado nuevo
reason	text nullable	Motivo del cambio
changed_by	FK (users) nullable	Usuario que hizo el cambio
changed_at	timestamp	Fecha del cambio

Tabla: `user_roles` (Roles asignados a usuarios)

Campo	Tipo	Descripción
id	bigint PK	Identificador
user_id	FK (cascade)	Usuario local
role_id	FK (cascade)	Rol (de tabla central `roles`)
unique(user_id, role_id)	—	Un usuario no puede tener el mismo rol dos veces

Tabla: `activity_log` (Auditoría — Spatie)

Campo	Tipo	Descripción
id	bigint PK	Identificador
log_name	string nullable	Nombre del log
description	text	Descripción de la acción
subject_type/id	morphs	Entidad afectada (ej: Member)
causer_type/id	morphs nullable	Usuario que realizó la acción
properties	JSON	Datos old/new del cambio
event	string nullable	Tipo de evento
batch_uuid	UUID nullable	Agrupación de cambios

---

8. Core SaaS (Plataforma)

El Core SaaS gestiona la plataforma misma. Vive en `app/Core/` y opera sobre la BD central.

Core/Tenants — Gestión de Iglesias

Archivo	Función

Archivo	Función
`Domain/Models/Tenant.php`	Modelo que extiende `BaseTenant` de stancl. Campos: church_name, slug, plan_id, status, pastor_name, email, phone, address. Método `hasModule(slug)`: verifica si el plan del tenant incluye un módulo.
`Infrastructure/Providers/TenantsServiceProvider.php`	Service provider del módulo

Core/Plans — Planes de Suscripción

Archivo	Función
`Domain/Models/Plan.php`	Modelo con campos: name, slug, price, billing_cycle, max_members, features (JSON), is_active. Relación `belongsToMany(Module)`. Método `hasModule(slug)`.
`Infrastructure/Providers/PlansServiceProvider.php`	Service provider del módulo

Core/Settings — Configuración Global

Archivo	Función
`Domain/Models/Setting.php`	Modelo clave-valor con caché Redis (1 hora). Métodos estáticos: `get(key, default)`, `set(key, value)`, `clearCache()`.
`Infrastructure/Providers/SettingsServiceProvider.php`	Service provider del módulo

Core/Billing — Facturación (esqueleto)

Archivo	Función
`Infrastructure/Providers/BillingServiceProvider.php`	Estructura lista para implementar cobros y facturación

Service Providers Centrales

Provider	Función
`AppServiceProvider.php`	Carga migraciones shared (`database/migrations/shared`)
`ModuleServiceProvider.php`	Registra los 4 Core providers + 9 Module providers
`TenancyServiceProvider.php`	Configura eventos de stancl/tenancy: crear BD, migrar, seed
`Filament/AdminPanelProvider.php`	Configura panel Filament: colores, login, navegación

9. Sistema Modular (Negocio)

Cada módulo de negocio vive en `app/Modules/` y opera sobre la BD del tenant. Siguen un patrón DDD Lite:

```
Módulo/
├── Application/      ← Casos de uso, DTOs, Form Requests
│   ├── Actions/
│   ├── DTOs/
│   └── Requests/
├── Domain/           ← Modelos Eloquent, eventos de dominio, servicios
│   ├── Events/
│   ├── Models/
│   └── Services/
├── Infrastructure/   ← Providers, repositorios, persistencia
│   ├── Providers/
│   └── Repositories/
└── Presentation/     ← Controllers (API/Web), Resources
    ├── Controllers/
    └── Resources/
```

Los 9 Módulos

#	Módulo	Slug	Función	Estado
1	**Church**	`church`	Configuración y datos de la iglesia	✅ Básico
2	**Members**	`members`	Gestión de miembros y familias	✅ Completo
3	**Groups**	`groups`	Células, ministerios, equipos de servicio	📋 Esqueleto
4	**Attendance**	`attendance`	Registro de asistencia a servicios y eventos	📋 Esqueleto
5	**Activities**	`activities`	Calendario de eventos y actividades	📋 Esqueleto
6	**Finance**	`finance`	Diezmos, ofrendas, gastos, presupuestos	📋 Esqueleto
7	**Communication**	`communication`	Envío de emails y notificaciones masivas	📋 Esqueleto
8	**Projects**	`projects`	Gestión de proyectos de la iglesia	📋 Esqueleto
9	**Reports**	`reports`	Reportes y estadísticas	📋 Esqueleto

Control de Acceso por Módulo

El middleware `CheckModuleAccess` valida que el tenant tenga acceso al módulo:

```php
Route::middleware('module:finance')->group(function() {
    // Solo accesible si el plan del tenant incluye el módulo 'finance'
});
```

Flujo: `request → CheckModuleAccess → tenant()->hasModule(slug) → plan->hasModule(slug) → plan_modules table`

Módulo Members (detalle de implementación)

Es el primer módulo completamente implementado. Sirve como referencia para los demás:

Backend:

- Modelo `Member.php` con 30+ campos, soft deletes, audit logging automático
- Modelo `Family.php` para agrupación familiar
- Modelo `MemberSkill.php` para habilidades/dones
- Modelo `MemberHistory.php` para historial de cambios de estado
- `MemberController.php` con CRUD completo (search, filter, pagination)
- 5 migraciones para el schema completo

Frontend:

- `Members/Index.tsx` — Tabla con búsqueda, filtros por status/categoría, paginación, stats cards
- `Members/Create.tsx` / `Edit.tsx` — Formulario multi-sección reutilizable
- `Members/Show.tsx` — Perfil con tabs (info, espiritual, familia, historial)
- `MemberForm.tsx` — Componente de formulario compartido (5 secciones)
- Tipos TypeScript completos en `types/members.ts`

---

10. Panel Super Admin (Filament)

Acceso

- **URL:** `https://poimano.com/admin`
- **Usuario:** `admin@poimano.com`
- **Framework:** Filament v3.3 (Livewire + Alpine.js)
- **Solo accesible desde dominios centrales**

Recursos (CRUD)

TenantResource — Gestión de Iglesias

Sección del Form	Campos	Descripción
Información de la Iglesia	church_name → auto-genera slug e id, plan_id (select), status	Datos principales
Contacto del Pastor	pastor_name, email, phone, address	Pastor principal
Credenciales Admin	admin_password (visible solo al crear)	Contraseña del admin de la iglesia

Tabla: church_name, slug (copiable), pastor_name, email, plan badge, status badge, fecha

Vista detalle (Infolist): Todas las secciones en modo lectura

PlanResource — Gestión de Planes

Sección del Form	Campos	Descripción
Información	name → auto-genera slug, description	Datos del plan
Precios y Límites	price, billing_cycle, max_members, is_active	Config de facturación
Features	KeyValue field (JSON)	Características incluidas

Widgets del Dashboard

Widget	Función
`StatsOverview`	Cards con KPIs: total iglesias, activas, en prueba
`LatestTenants`	Tabla con las últimas iglesias registradas
`TenantsChart`	Gráfico de evolución de iglesias en el tiempo

Personalización Visual

- **Colores:** Navy (`#00105E`) como primario, Cyan (`#00E1FF`) como info
- **Login personalizado:** `app/Filament/Pages/Auth/Login.php`
- **Settings UI:** `app/Filament/Pages/PlatformSettings.php`

---

11. Frontend Tenant (React + Inertia)

Arquitectura Frontend

```
resources/js/
├── app.tsx              ← Entry point: React 19 + Inertia setup + page resolver
├── bootstrap.js         ← Axios defaults
├── vite-env.d.ts        ← Vite types
├── lib/
│   └── utils.ts         ← cn() helper (clsx + tailwind-merge)
├── types/
│   ├── index.ts         ← PageProps, User, Flash (Inertia shared)
│   └── members.ts       ← Member, Family, MemberSkill, enums, label maps
├── layouts/
│   └── TenantLayout.tsx ← Sidebar, header, nav, flash messages
├── components/
│   ├── ui/              ← 15+ componentes shadcn/ui
│   │   ├── button.tsx, card.tsx, input.tsx, label.tsx
│   │   ├── select.tsx, dialog.tsx, alert-dialog.tsx
│   │   ├── dropdown-menu.tsx, tabs.tsx, tooltip.tsx
│   │   ├── table.tsx, badge.tsx, avatar.tsx
│   │   ├── separator.tsx, spinner.tsx
│   │   └── index.ts    ← Barrel export
│   └── members/
│       └── MemberForm.tsx ← Formulario reutilizable (5 secciones)
```

```
└── pages/
    ├── Auth/Login.tsx      ← Login form
    ├── Dashboard.tsx       ← Dashboard con stats placeholder
    └── Members/
        ├── Index.tsx       ← Lista + búsqueda + filtros + paginación + delete
        ├── Create.tsx      ← Crear miembro
        ├── Edit.tsx        ← Editar miembro
        └── Show.tsx        ← Perfil detallado con tabs
```

TenantLayout — Layout Principal

- **Sidebar colapsable** con navegación por módulo
- **Header** con nombre de iglesia + botón logout
- **Flash messages** (success/error) con auto-dismiss
- **Navegación:** Dashboard, Miembros, Grupos, Actividades, Proyectos, Finanzas, Comunicación, Reportes
- **Active state:** Detecta ruta actual con `startsWith` para nested routes

Componentes UI (shadcn/ui)

Basados en Radix UI primitives. Estilizados con Tailwind CSS + `class-variance-authority` para variants.

Componente	Uso en el proyecto
`Button`	Acciones, submit, navegación
`Card`	Contenedores de secciones
`Input` / `Label`	Campos de formulario
`Select`	Dropdowns de selección
`Table`	Tablas de datos (Members Index)
`Dialog`	Modales de confirmación
`AlertDialog`	Confirmación destructiva (delete)
`Tabs`	Navegación por secciones (Members Show)
`DropdownMenu`	Menú de acciones por fila
`Badge`	Status tags (Activo, Bautizado, etc.)
`Avatar`	Fotos de perfil de miembros
`Tooltip`	Información contextual
`Spinner`	Loading states

Tema y Estilos

- **Font:** Inter (sans-serif)
- **Colores brand:** Navy `#00105E`, Cyan `#00E1FF`
- **Dark mode:** Soporte via clase CSS (desactivado por defecto)

- **CSS Variables:** Sistema completo para tema: `--background`, `--foreground`, `--primary`, `--secondary`, `--destructive`, `--muted`, `--accent`, etc.
- **Plugin:** `tailwindcss-animate` para transiciones

---

12. Autenticación y Autorización

Autenticación Central (Super Admin)

Aspecto	Implementación
Guard	`web` (default Laravel)
Modelo	`App\Models\User` (BD central)
UI	Filament Login page customizada
Validación	`canAccessPanel()` → `role === 'admin' || is_admin`
Credencial	`admin@poimano.com` / `Admin2026`

Autenticación Tenant (Iglesia)

Aspecto	Implementación
Guard	`web` (dentro del tenant context — BD tenant)
Modelo	`App\Models\User` (BD tenant)
Controller	`AuthController::login()`
UI	React `Auth/Login.tsx` via Inertia
Validación	`Auth::attempt([email, password, is_active => true])`
Auto-generado	Al crear iglesia: email del pastor + password del formulario

RBAC (Role-Based Access Control)

6 Roles pre-configurados por tenant:

Rol	Alcance	Permisos
Pastor	Total	Todos los permisos de todos los módulos
Administrador	Total	Todos los permisos de todos los módulos
Líder	Parcial	Members (CRUD), Groups (CRUD), Attendance (create), Activities (CRUD), Church (view)
Tesorero	Financiero	Finance (CRUD), Reports (view+export), Church (view)
Secretaria	Administrativo	Members (CRUD+export), Groups/Attendance/Activities (view+create), Reports, Church (view)
Miembro	Lectura	Members/Groups/Attendance/Activities/Church (solo view)

30+ Permisos granulares por módulo:

- `members.view`, `members.create`, `members.edit`, `members.delete`, `members.export`
- `groups.view`, `groups.create`, `groups.edit`, `groups.delete`
- `attendance.view`, `attendance.create`
- `finance.view`, `finance.create`, `finance.edit`, `finance.delete`
- `communication.view`, `communication.create`
- `activities.view`, `activities.create`, `activities.edit`, `activities.delete`
- `projects.view`, `projects.create`, `projects.edit`, `projects.delete`
- `reports.view`, `reports.export`
- `church.view`, `church.edit`

Control de Acceso por Módulo (Plan)

Independiente del RBAC. Controla qué módulos puede ver cada iglesia según su plan:

Plan	Precio	Máx. Miembros	Módulos
Gratuito	$0/mes	50	Church, Members, Groups, Attendance
Esencial	$29.99/mes	200	+Finance, Communication, Reports
Profesional	$59.99/mes	1,000	+Activities, Projects
Enterprise	$149.99/mes	10,000	Todos los módulos

---

13. Rutas del Sistema

Rutas Centrales (`routes/web.php`)

Método	URI	Controlador/Vista	Descripción
GET	`/`	`welcome.blade.php`	Landing page pública
—	`/admin/*`	Filament (auto-registrado)	Panel de super administración

Rutas Tenant (`routes/tenant.php`)

Middleware stack: `web` + `InitializeTenancyBySubdomain` + `PreventAccessFromCentralDomains`

Rutas Públicas (Guest)

Método	URI	Controlador	Descripción
GET	`/login`	`AuthController@showLogin`	Página de login
POST	`/login`	`AuthController@login`	Procesar login

Rutas Protegidas (Auth)

Método	URI	Controlador	Descripción

Método	URI	Controlador	Descripción
GET	`/dashboard`	Inertia `Dashboard`	Dashboard principal
POST	`/logout`	`AuthController@logout`	Cerrar sesión
GET	`/members`	`MemberController@index`	Listar miembros
GET	`/members/create`	`MemberController@create`	Form crear miembro
POST	`/members`	`MemberController@store`	Guardar miembro
GET	`/members/{member}`	`MemberController@show`	Ver detalle miembro
GET	`/members/{member}/edit`	`MemberController@edit`	Form editar miembro
PUT	`/members/{member}`	`MemberController@update`	Actualizar miembro
DELETE	`/members/{member}`	`MemberController@destroy`	Eliminar miembro (soft)

14. Seeders y Datos Iniciales

Seeders Centrales

Seeder	Datos que crea
`SuperAdminSeeder`	Usuario `admin@poimano.com` con role `admin`
`PlanSeeder`	4 planes: Gratuito, Esencial, Profesional, Enterprise
`ModuleSeeder`	9 módulos + matriz de asignación plan↔módulo

Seeders Tenant (ejecutados automáticamente al crear iglesia)

Seeder	Orden	Datos que crea
`RolesSeeder`	1°	6 roles (Pastor→Miembro) + 30+ permisos + asignaciones rol↔permiso
`ChurchSettingsSeeder`	2°	Configuración inicial de la iglesia (nombre, colores, timezone, moneda)
`AdminUserSeeder`	3°	Usuario admin local con email del tenant y password `Poimano2026!`

> **Nota:** El `afterCreate()` de `CreateTenant.php` sobreescribe la contraseña del admin con la ingresada en el formulario de Filament.

15. Estado Actual por Módulo

Resumen de Progreso

```
~25% completado (2/9 módulos + plataforma)
```

Área	Estado	Detalle
Infraestructura	✅ Completo	VPS, dominio, SSL wildcard, Nginx, PostgreSQL, deploy
Multi-Tenancy	✅ Completo	DB-per-tenant, subdomain identification, auto-migrate/seed
Panel Super Admin	✅ Completo	CRUD Iglesias, CRUD Planes, Widgets dashboard, Login custom
Auth Tenant	✅ Completo	Login/logout, sesiones, flash messages
Frontend Base	✅ Completo	Layout, shadcn/ui (15 components), tipos, tema
Módulo Church	✅ Básico	Modelo + seeder + migración (falta UI de settings)
Módulo Members	✅ Completo	Backend + Frontend CRUD completo + auditoría
Módulo Groups	📋 Esqueleto	Estructura DDD + migración vacía
Módulo Attendance	📋 Esqueleto	Estructura DDD + migración vacía
Módulo Activities	📋 Esqueleto	Estructura DDD + migración vacía
Módulo Finance	📋 Esqueleto	Estructura DDD + migración vacía
Módulo Communication	📋 Esqueleto	Estructura DDD + migración vacía
Módulo Projects	📋 Esqueleto	Estructura DDD + migración vacía
Módulo Reports	📋 Esqueleto	Estructura DDD + migración vacía
API REST	⏳ Pendiente	Para futura app Flutter
Testing	⏳ Pendiente	PHPUnit + Jest configurados pero sin tests
Billing real	⏳ Pendiente	Integración con pasarela de pagos

Commits Clave

Commit	Descripción
`64119aa`	Members module backend (migrations, models, RBAC, audit log)
`71c0285`	Members UI completo (Index/Create/Edit/Show + controller + routes)
`0fea499`	Traducciones al español (validation, auth, passwords, pagination)
`13c1db6`	Fix 500 errors (central_domains, AdminUserSeeder, CreateTenant duplicate)
`a1a43bc`	Fix admin_password dehydration, canAccessPanel, robust afterCreate

16. Estructura de Archivos

```
poimano/
│
├── app/
│   ├── Console/Commands/
│   │   └── CreateTenantCommand.php          # CLI: php artisan tenant:create
│   │
│   ├── Core/                                # ── PLATAFORMA SAAS ──
│   │   ├── Billing/Infrastructure/Providers/
│   │   ├── Plans/
│   │   │   ├── Domain/Models/Plan.php
│   │   │   └── Infrastructure/Providers/
│   │   ├── Settings/
│   │   │   ├── Domain/Models/Setting.php
│   │   │   └── Infrastructure/Providers/
│   │   └── Tenants/
│   │       ├── Domain/Models/Tenant.php
│   │       └── Infrastructure/Providers/
│   │
│   ├── Modules/                             # ── MÓDULOS DE NEGOCIO ──
│   │   ├── Activities/  (esqueleto DDD)
│   │   ├── Attendance/  (esqueleto DDD)
│   │   ├── Church/
│   │   │   └── Domain/Models/ChurchSetting.php
│   │   ├── Communication/ (esqueleto DDD)
│   │   ├── Finance/     (esqueleto DDD)
│   │   ├── Groups/      (esqueleto DDD)
│   │   ├── Members/
│   │   │   └── Domain/Models/
│   │   │       ├── Member.php
│   │   │       ├── Family.php
│   │   │       ├── MemberSkill.php
│   │   │       └── MemberHistory.php
│   │   ├── Projects/    (esqueleto DDD)
│   │   └── Reports/     (esqueleto DDD)
│   │
│   ├── Models/                              # ── MODELOS COMPARTIDOS ──
│   │   ├── User.php
│   │   └── Shared/
│   │       ├── Role.php
│   │       ├── Permission.php
│   │       ├── Module.php
│   │       ├── Country.php
│   │       └── Template.php
│   │
│   ├── Http/
│   │   ├── Controllers/Tenant/
│   │   │   ├── AuthController.php
│   │   │   └── MemberController.php
│   │   └── Middleware/
```

```
│   │       ├── HandleInertiaRequests.php
│   │       └── CheckModuleAccess.php
│   │
│   ├── Filament/                           # ── PANEL SUPER ADMIN ──
│   │   ├── Pages/
│   │   │   ├── Auth/Login.php
│   │   │   └── PlatformSettings.php
│   │   ├── Resources/
│   │   │   ├── TenantResource.php
│   │   │   ├── TenantResource/Pages/ (List, Create, Edit, View)
│   │   │   ├── PlanResource.php
│   │   │   └── PlanResource/Pages/ (List, Create, Edit)
│   │   └── Widgets/
│   │       ├── StatsOverview.php
│   │       ├── LatestTenants.php
│   │       └── TenantsChart.php
│   │
│   └── Providers/
│       ├── AppServiceProvider.php
│       ├── ModuleServiceProvider.php
│       ├── TenancyServiceProvider.php
│       └── Filament/AdminPanelProvider.php
│
├── database/
│   ├── migrations/
│   │   ├── (central – Laravel + stancl/tenancy + plans + settings)
│   │   ├── shared/ (countries, roles, permissions, templates, modules)
│   │   └── tenant/
│   │       ├── (users, activity_log)
│   │       ├── church/ (church_settings)
│   │       └── members/ (families, members, member_skills, member_history,
user_roles)
│   └── seeders/
│       ├── Central/ (SuperAdmin, Plans, Modules)
│       └── Tenant/ (TenantDatabaseSeeder, Roles, ChurchSettings, AdminUser)
│
├── resources/
│   ├── css/ (app.css + filament/)
│   ├── js/
│   │   ├── app.tsx
│   │   ├── layouts/TenantLayout.tsx
│   │   ├── components/
│   │   │   ├── ui/ (15+ shadcn components)
│   │   │   └── members/MemberForm.tsx
│   │   ├── pages/
│   │   │   ├── Auth/Login.tsx
│   │   │   ├── Dashboard.tsx
│   │   │   └── Members/ (Index, Create, Edit, Show)
│   │   ├── types/ (index.ts, members.ts)
│   │   └── lib/utils.ts
│   └── views/
│       ├── app.blade.php (root Inertia template)
│       └── welcome.blade.php (landing page)
│
```

```
├── routes/
│   ├── web.php (central)
│   ├── tenant.php (tenant - auth + members)
│   └── console.php
│
├── lang/es/
│   ├── validation.php
│   ├── auth.php
│   ├── passwords.php
│   └── pagination.php
│
├── config/tenancy.php
├── composer.json
├── package.json
├── vite.config.js
├── tsconfig.json
└── tailwind.config.js
```

---

17. Flujos Críticos

Flujo 1: Crear una nueva iglesia

```
Super Admin → Filament /admin/tenants/create
  ↓
Llena formulario: church_name, slug, plan, pastor_name, email, admin_password
  ↓
mutateFormDataBeforeCreate() → extrae admin_password del form data
  ↓
Filament crea registro Tenant en BD central
  ↓
Evento TenancyCreated se dispara automáticamente
  ├─ CreateDatabase    → CREATE DATABASE tenant_{slug}
  ├─ MigrateDatabase   → Ejecuta todas las migraciones de /tenant/*
  └─ SeedDatabase      → Ejecuta TenantDatabaseSeeder
       ├─ RolesSeeder          → 6 roles + 30+ permisos
       ├─ ChurchSettingsSeeder → Config inicial iglesia
       └─ AdminUserSeeder      → User admin (password: Poimano2026!)
  ↓
afterCreate() en CreateTenant.php
  ├─ Crea dominio: tenant.domains()->firstOrCreate({domain: slug})
  └─ En el contexto del tenant:
       ├─ Busca usuario por email del pastor
       ├─ Si existe → actualiza password con la del formulario
       └─ Si no existe → crea User con password del formulario
  ↓
Redirect a lista de iglesias con notificación de éxito
```

Flujo 2: Login en subdominio tenant

```
Usuario navega a iglesia1.poimano.com
  ↓
Nginx → Laravel → Middleware InitializeTenancyBySubdomain
  ↓
Extrae "iglesia1" del host → Busca en tabla domains → Inicializa tenant
  ↓
Middleware PreventAccessFromCentralDomains → OK (no es dominio central)
  ↓
Middleware auth → No autenticado → Redirect /login
  ↓
AuthController@showLogin → Inertia::render('Auth/Login')
  ↓
React renderiza Login.tsx con formulario
  ↓
POST /login → AuthController@login
  ├─ Valida email + password
  ├─ Auth::attempt([email, password, is_active => true])
  ├─ ✅ Éxito → regenerate session → redirect /dashboard
  └─ ❌ Fallo → back()->withErrors('Las credenciales no coinciden...')
```

Flujo 3: CRUD de Miembros

```
/members (GET) → MemberController@index
  ├─ Query: search (ilike en nombre, email, phone)
  ├─ Filtros: status, category
  ├─ Stats: total, activos, bautizados, nuevos (último mes)
  ├─ Paginación: 15 por página
  └─ Inertia::render('Members/Index', {members, filters, stats, ...})

/members/create (GET) → MemberController@create
  └─ Carga: families, members (para referred_by)

/members (POST) → MemberController@store
  ├─ Validación: 30+ campos con reglas
  ├─ Member::create($validated)
  └─ Redirect /members con flash success

/members/{id} (GET) → MemberController@show
  └─ Carga: family.members, skills, history, referrals, referredBy

/members/{id}/edit (GET) → MemberController@edit
  └─ Mismos datos que create + member actual

/members/{id} (PUT) → MemberController@update
  └─ Validación + update + redirect con flash

/members/{id} (DELETE) → MemberController@destroy
  └─ Soft delete + redirect con flash
```

18. Roadmap Pendiente

Prioridad Alta (Funcionalidad Core)

Tarea	Descripción
Módulo Groups	CRUD de células/ministerios + asignación de miembros + líderes
Módulo Attendance	Registro de asistencia a servicios + reportes automáticos
Módulo Finance	Diezmos, ofrendas, gastos + reportes financieros
Church Settings UI	Pantalla para que la iglesia configure su info, logo, colores

Prioridad Media (Experiencia)

Tarea	Descripción
Módulo Activities	Calendario de eventos + inscripciones
Módulo Communication	Envío de emails/notificaciones masivas a miembros
Dashboard dinámico	Charts y KPIs reales (Recharts ya instalado)
Perfil de usuario	Editar datos propios + cambiar contraseña

Prioridad Baja (Escalamiento)

Tarea	Descripción
Módulo Projects	Gestión de proyectos de la iglesia
Módulo Reports	Reportes PDF/Excel exportables
API REST	Endpoints para futura app Flutter
Testing	PHPUnit (backend) + Jest/Vitest (frontend)
Billing real	Integración pasarela de pagos (Stripe / PayU)
Redis/Horizon	Colas asíncronas para comunicaciones masivas
Observability	Laravel Pulse / Telescope para monitoreo

> **Documento generado automáticamente — Estado al 14 de marzo de 2026** Repositorio: https://github.com/yohanrojas562-code/poimano